AWS VPC Peering Project

EWB Cloud Labs - Student Project Report

Field	Details
Name	Anirudh A Zalki
Course	AWS Cloud Computing
Batch	Evening 1
Project	VPC Peering: connect two VPCs and prove private connectivity
Region	Asia Pacific (Sydney) - ap-southeast-2
Completion date	25-10-2026

1. Architecture and inventory

Architecture: a laptop connects over SSH (port 22) to the bastion in the public subnet of VPC A (10.10.0.0/16) through the internet gateway ewb-igw-a. From the bastion, SSH reaches App-A (private subnet A) and, through the peering connection ewb-a-b-peer, App-B in VPC B (10.20.0.0/16). App-A and App-B communicate over private IPs through the peering connection. Insert your architecture diagram image here. Fill in the blank IDs from the console (Peering connections, Route tables, Security groups, Instances).

Resource	Name	ID / Address	CIDR / IP
VPC A	ewb-vpc-a	vpc-02cade71661e2f9d2	10.10.0.0/16
VPC B	ewb-vpc-b	vpc-0ea7ea17238f4ad89	10.20.0.0/16
Public subnet A	ewb-a-public	subnet-045d42dc88c56adc1	10.10.1.0/24
Private subnet A	ewb-a-private	subnet-026b51d5e1b785e74	10.10.2.0/24
Private subnet B	ewb-b-private	subnet-_____	10.20.1.0/24
Internet gateway	ewb-igw-a	igw-04b72aa21c4ec7e68	n/a
Route table	ewb-rt-a-public	rtb-08571f3a7a2f5dc9a	n/a
Route table	ewb-rt-a-private	rtb-008242c681f603100	n/a
Route table	ewb-rt-b-private	rtb-0d7513d84772da3ab	n/a
Peering	ewb-a-b-peer	pcx-08bb1f167f32b5863	n/a
Security group	ewb-sg-bastion	sg-0127bc7af42abadc1	n/a
Security group	ewb-sg-app-a	sg-0275d701a29e10e1a	n/a
Security group	ewb-sg-app-b	sg-0a86c60dfafaf81db	n/a
EC2 instance	ewb-bastion	i-09c89e4f3a44bda1a	10.10.1.10 + public IP
EC2 instance	ewb-app-a	i-0f8b306e3efd98a30	10.10.2.10 (no public IP)
EC2 instance	ewb-app-b	i-0f0a66f4d202e83cb	10.20.1.10 (no public IP)

Note: the bastion's public IPv4 changes if the instance is stopped and started. Record the value you used for each test.

2. Evidence checklist

Label every screenshot with the host, the test and the result. Do not include private keys, passwords or tokens.

#	Evidence (label host, test, result)	Done
1	Subnet list showing all three CIDRs	[done]
2	ewb-rt-a-public: routes + subnet association	[done]
3	ewb-rt-a-private: routes + subnet association	[done]
4	ewb-rt-b-private: routes + subnet association	[done]
5	Peering connection Active, both VPC IDs visible	[done]
6	Inbound rules: ewb-sg-bastion	[done]
7	Inbound rules: ewb-sg-app-a	[done]
8	Inbound rules: ewb-sg-app-b	[done]
9	Baseline: App-A ping 10.20.1.10 (100% loss)	[done]
10	Forward: App-A ping 10.20.1.10 (0% loss)	[done]
11	Reverse: App-B ping 10.10.2.10 (0% loss)	[done]
12	SSH to App-B through the bastion	[done]
13	App-A and App-B have no public IPv4	[done]
14	Controlled failure: ICMP rule removed, ping fails	[done]
15	Recovery: rule restored, ping works	[done]
16	Cleanup: tag-filtered check of EC2, volumes, VPC resources	[done]

3. Forward and return path explanation

- Forward path: App-A (10.10.2.10) sends to 10.20.1.10. ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection into VPC B.
- Return path: App-B replies to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and sends it back through the peering connection.
- Security groups are stateful, so replies to permitted ICMP and SSH are allowed automatically.

4. Viva questions - write your answers

1. Why is the return route needed?

Peering only creates the link. Each VPC's route table still decides where its own traffic goes. When App-B replies to App-A (10.10.2.10), the subnet route table of VPC B must have 10.10.0.0/16 pointing to the peering connection. Without that route the reply has no path back, so the request arrives but the ping or SSH session never completes.

2. Why is the App-B SSH source the bastion's private IP?

With ProxyJump, the bastion opens a separate SSH connection to App-B. App-B therefore sees the bastion's private address (10.10.1.10) as the source, not my laptop's public IP. So ewb-sg-app-b allows TCP 22 only from 10.10.1.10/32.

3. Why does Active peering not guarantee a successful ping?

Active only means the two VPCs are linked. A ping also needs routes in both directions (peer routes in the subnet route tables) and a security group rule that allows ICMP Echo request on the destination. If any of these is missing the ping fails, even though the peering shows Active. Both outbound access on the sending host and the inbound rule on the receiving host must also allow the traffic.

4. Can VPC B use VPC A's internet gateway over peering?

No. VPC peering is non-transitive and does not forward traffic to an internet gateway in the other VPC. VPC B has no internet gateway or route to one, so its instances stay private and have no internet access through VPC A.

5. Controlled failure and recovery

I removed only the Custom ICMP Echo request rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. From App-A, ping -c 4 10.20.1.10 then showed 100% packet loss, even though the peering and routes were still in place. This proves the security group controls access to the destination. I then restored the exact rule (Custom ICMP - IPv4, Echo Request, source 10.10.2.10/32) and the ping replies resumed with 0% loss.

6. Cleanup checklist

Step	Action	Done
1	Terminate ewb-bastion, ewb-app-a, ewb-app-b; confirm volumes deleted	[ ]
2	Delete peer routes from the three route tables, then delete ewb-a-b-peer	[ ]
3	Delete security groups ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b	[ ]
4	Delete route tables ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private	[ ]
5	Delete the three subnets	[ ]
6	Detach and delete ewb-igw-a	[ ]
7	Delete ewb-vpc-a and ewb-vpc-b	[ ]
8	Delete key pair ewb-lab and the local .pem files	[ ]
9	Verify by Project tag in ap-southeast-2; check Billing	[ ]

7. What I learned

This project taught me how two private networks can talk to each other without the internet. I learned that a VPC peering connection must be requested and accepted, and that it only works once both sides have routes to the other VPC's CIDR, which is why overlapping CIDRs are not allowed. I saw that security groups are stateful but still need the right inbound rule on the destination, and that a missing outbound rule on a host stops it from starting traffic. Using a bastion host with SSH ProxyJump showed me how to reach private servers without giving them public IPs or copying keys to them. Finally, the controlled failure test showed how one rule change can break and then restore connectivity, and the cleanup steps taught me to remove resources in the right order to avoid charges.